

Partitioned Hash Join with Duplicates

An OpenMP Study

Gabriele Marsili

SPM a.y. 2025/26

1 Introduction

This report describes the OpenMP implementation of the partitioned hash join developed in Module 2. The algorithm is unchanged: R and S are partitioned through a common key mapping and a local join is performed independently on each partition. The objective of the study is to assess which OpenMP constructs are appropriate for this kernel under both balanced and skewed workloads, and to compare the resulting performance with the `std::thread` implementation of Module 2.

Two parallel variants were developed. One uses loop-level parallelism through a `parallel for` on the join phase; the other emits explicit tasks ordered by Longest Processing Time First. The partitioning code is shared between the two, which makes the comparison between OpenMP constructs independent of the upstream phases. All experiments were carried out on a node of the SPM cluster.

2 Algorithm and OpenMP Strategies

2.1 Three-phase Structure

Given relations R and S of size N_R and N_S , each tuple carries a key k which is mapped to partition $p = k \bmod P$ with $P = 128$. Since P is a power of two, the equivalent bitmask form $k \& (P - 1)$ is used. Each relation is processed in three phases:

1. **Histogram** – count tuples per partition.
2. **Scatter** – compute exclusive prefix-sum offsets and write tuples into contiguous per-partition buffers.
3. **Join** – on each partition p , build a hash table on R_p and probe it with the tuples of S_p .

The per-partition hash table is the `FlatCountMap` introduced and analysed in the Module 2 report: open-addressing with linear probing, count-at-build, multiplicity accumulation at probe time. Its sizing rule and load-factor bound are unchanged here, so we do not repeat them and refer to the Module 2 report for the details.

2.2 Loop-level Variant

The loop variant follows the canonical `parallel for` pattern. Each thread maintains a private histogram in a 64-byte aligned buffer; the merge into the global histogram

and the exclusive prefix sum are executed by a single thread inside an `#pragma omp single` block. The scatter phase is a `parallel for` with `schedule(static)`, since the per-tuple work is constant. The join phase is then expressed as

```
#pragma omp parallel for schedule(dynamic,1) \  
    reduction(+: join_count, c1, c2)  
for (uint32_t p = 0; p < P; ++p)  
    join_one_partition(R[p], S[p], ...);
```

The choice of `dynamic,1` follows from the workload-balancing argument of lecture 14: when the per-iteration cost is irregular and unknown ahead of time, a static distribution leaves the slowest worker on the critical path, while a dynamic policy reassigns chunks on demand and keeps idle threads engaged. Under the skew of Section 3 the per-partition cost varies by two orders of magnitude, so the static schedule is dominated by the makespan of the heaviest chunk; chunk size 1 minimises the residual within-chunk imbalance at the cost of a small amount of runtime bookkeeping. The schedule sweep of Section 5.5 confirms that this overhead is negligible at $P/T = 16$. The `reduction(+:...)` clause is sufficient because only three `uint64_t` scalars are accumulated.

2.3 Task Variant with LPT

The task variant differs from the loop variant only in the join. The histogram and the scatter run on the same `#pragma omp for` construct in both versions because their per-element work is uniform by construction: each tuple costs a hash, an optional prefetch, and one indexed read/write. Lecture 14 distinguishes regular workloads, for which a static work-sharing partition is sufficient, from irregular ones, where dynamic assignment becomes worthwhile. The partitioning phases fall in the first class, so wrapping them in tasks would add submission and queueing overhead without any balancing benefit, and would also fragment the implicit barrier on which the scatter relies to read its inputs. The task machinery is therefore reserved for the join, where the per-partition cost is genuinely irregular. The join is expressed as an explicit task loop:

```
#pragma omp parallel default(none) \  
    shared(P,order,R,S,thr_results)  
{  
    #pragma omp single nowait  
    for (uint32_t i = 0; i < P; ++i) {  
        uint32_t pid = order[i];  
        #pragma omp task firstprivate(pid) \  
            shared(R, S, thr_results)
```

```

    { ... }
  }
}

```

The `nowait` clause is required: without it the $T - 1$ workers would wait at the implicit barrier of `single` until task emission completes, instead of consuming tasks while the master is still creating them.

The `order` array drives the scheduling. Partitions are sorted by decreasing $|R_p| + |S_p|$ before submission, which is the Longest Processing Time First (LPT) heuristic. The course material covers dynamic assignment for irregular workloads (lez. 14) but does not prescribe a specific submission order, so the policy was chosen by looking at the scheduling literature outside the lectures. LPT is the classical greedy for makespan minimisation when independent tasks of variable size are dispatched to identical workers, and on a problem of this granularity ($P = 128$ partitions, $T \leq 32$) it has two practical effects: heavy partitions enter the queue first, so they are unlikely to land on the critical path at the end of the run, and small partitions remain available to fill the gaps left by the heavy ones. The sum $|R_p| + |S_p|$ is an appropriate cost proxy because the per-partition cost is linear in the size of its two inputs.

All tasks are tied, which is the OpenMP default. The accumulation into `thr_results` reads `omp_get_thread_num()` and uses the returned index as the position of the per-thread slot. Tied semantics keep that index valid for the entire body of a task even in the presence of task scheduling points; this is relevant in the hot-partition path, where the outer task contains an inner `taskgroup` that could otherwise expose the body to a thread switch on resumption. Untied tasks would offer no advantage here and would require careful reasoning around the same TSP, so the default is retained.

The reduction uses a manually padded array `PaddedResult thr_results[T]`, declared with `alignas(64)` and validated by a `static_assert`, so per-thread slots sit on distinct cache lines and false sharing is removed.

2.4 Synchronisation and Memory Layout

Phase boundaries rely on the implicit barriers at the end of each worksharing construct, with one explicit `#pragma omp barrier` after the per-thread initialisation, since `local_hists[tid].assign(P,0)` occurs outside any worksharing region. All shared structures (histograms, accumulators) are aligned to cache-line boundaries to prevent false sharing.

2.5 Targeted Optimisations

Three additional optimisations were applied on top of the baseline strategy described above. They follow directly from the cost model of the kernel and from the structural bound observed on skewed input.

Software prefetching in scatter and probe. Both the scatter loop and the probe loop have a sequential read stream, which the hardware prefetcher already covers, and a random-access stream, which it

does not. Two `__builtin_prefetch` calls were inserted to mask the L1/L2 miss latency on these random accesses: in the scatter, the destination slot `part.data[cursors[tid][pid_next]]` is prefetched 12 iterations ahead; in the probe, the hash slot `tbl.slots[slot_of(k_next)]` is prefetched 8 iterations ahead. The distances are tuned to the latency of the L1/L2 path on the target node, leaving enough work between prefetch and use for the line to arrive.

Intra-partition parallelism for hot partitions.

The asymptotic skewed efficiency $\leq H/T$ is a structural bound: no external scheduling, including LPT, can extract more parallelism than the number of heavy partitions H . The bound is broken by exposing parallelism *within* a hot partition. The build phase still runs sequentially because concurrent updates on `FlatCountMap` would require atomic counters, but the probe is read-only on the table and parallelises naturally. A partition is classified as hot when its weight $|R_p| + |S_p|$ exceeds eight times the average; its probe is then split into K sub-tasks where $K = \min(T, \lceil w/\bar{w} \rceil)$, with $\bar{w}$ the average partition weight. The sub-tasks are emitted inside a `taskgroup` so that the shared table outlives all readers. The probe of each hot partition can therefore be served by up to T co-operating threads, which is what restores the parallelism that LPT alone could not extract.

NUMA placement of partition buffers. The partition output buffers `Rpart.data` and `Spart.data` are zero-initialised in parallel with `schedule(static)` before the partitioning runs. Under the Linux first-touch policy this places each page on the NUMA node of the thread that will later read and write it during the join. The input relations are still allocated single-threaded; this is the remaining gap noted in the discussion.

3 Skewed Workload

The skewed dataset is produced by a Bernoulli mixture model. With probability $\rho = 0.9$ the next key is sampled from a hot pool of $H = 4$ keys, chosen so that they fall on four distinct partitions; with probability $1 - \rho = 0.1$ the key is drawn uniformly from $[0, \text{max.key})$. The values $\rho = 0.9$ and $H = 4$ are the canonical heavy-hitter parameters from the hash-join skew literature (Balakesen *et al.*, ICDE 2013): the hot-to-cold ratio that follows is a consequence of those choices, not a tuning knob. The expected fraction of records assigned to a single hot partition is

$$f_{\text{hot}} = \frac{\rho}{H} + \frac{1-\rho}{P} = \frac{0.9}{4} + \frac{0.1}{128} \approx 0.226,$$

while a cold partition receives $f_{\text{cold}} \approx 0.00078$. The two values are a direct consequence of the choice of (ρ, H, P) and produce the order-of-magnitude gap between hot and cold partitions that the experiments below probe.

The generator is driven by a `splitmix64` RNG with the seed exposed on the command line, which makes runs reproducible bit-by-bit. The `hot_count` parameter

is clamped to P to prevent a non-terminating loop in the case where more hot partitions are requested than exist.

4 Validation

Two complementary mechanisms are used. For small inputs ($N_R, N_S \leq 500$) both `hashjoin_seq` and `hashjoin_omp` run an in-process *naive* $O(|R| \cdot |S|)$ verifier (`include/verifier.hpp`) that performs a double loop over all pairs and emits a `naive_verify=PASS` line when the triple matches the algorithm’s output; the verifier is invoked *outside* the timed region. For larger inputs the script `scripts/validate.sh` sweeps modes $\times$ workloads $\times$ thread counts $\in \{1, 2, 4, 8, 16\}$ at $N_R=10^6$, $N_S=2 \cdot 10^6$. The thread range stops at $T=16$ on purpose: once both sockets are engaged the parallel region, the task queue, and the reduction tree have been exercised in their full topology, so the additional points $T \in \{20, 32\}$ would only re-test the SMT layer without exposing new code paths. They are kept for the performance evaluation below, which spans the full $T \in \{1, 2, 4, 8, 16, 20, 32\}$ range. The script compares the triple (`join_count`, `checksum1`, `checksum2`) against a reference: the sequential baseline for uniform input and the parallel run at $T = 1$ in the same mode for skewed input (the sequential implementation does not generate skewed data). As an additional check, the loop and task variants at $T = 4$ are required to produce identical triples on the same input.

The checksums are constructed to be invariant under task ordering: each match contributes `splitmix64(k) · m` added modulo 2^{64} , so the total is independent of the order in which tasks complete. All combinations pass on the cluster.

5 Experimental Evaluation

5.1 Setup

The experiments are executed on one node of *spmcluster*: dual-socket Intel Xeon E5-2650 v3 at 2.30 GHz, 10 cores per socket, 2 hardware threads per core (40 logical CPUs over 20 physical cores), 128 GB RAM, two NUMA nodes. The build uses GCC 12.2 with OpenMP 4.5 and `-O3 -march=native`. Unless stated otherwise, the parameters are $N_R = 10^7$, $N_S = 2 \cdot 10^7$, $P = 128$, seed 42, with each cell averaged over three repetitions. Affinity is fixed via `OMP_PROC_BIND=close` and `OMP_PLACES=cores`.

5.2 Strong Scaling

Figure 1 reports the speedup at $T \in \{1, 2, 4, 8, 16, 20, 32\}$, covering the full physical-core range of the node ($T = 20$) and one operating point in the SMT range ($T = 32$). Total times are listed in Table 1.

On uniform input the loop and task variants are within $\sim 1\%$ at every thread count, which is the expected outcome rather than a negative result. The two share the same histogram and scatter implementations and differ only in the join dispatch; on independent tasks of *equal* cost `dynamic,1` and an LPT-ordered task pool reduce

T	loop unif.	task unif.	loop skew	task skew
1	0.477	0.467	0.445	0.444
2	0.272	0.268	0.244	0.243
4	0.144	0.142	0.151	0.130
8	0.085	0.084	0.102	0.085
16	0.063	0.064	0.092	0.080
20	0.062	0.062	0.091	0.080
32	0.065	0.066	0.093	0.081

Table 1: Mean total time [s], $N_R=10^7$, $N_S=2 \cdot 10^7$, $P=128$.

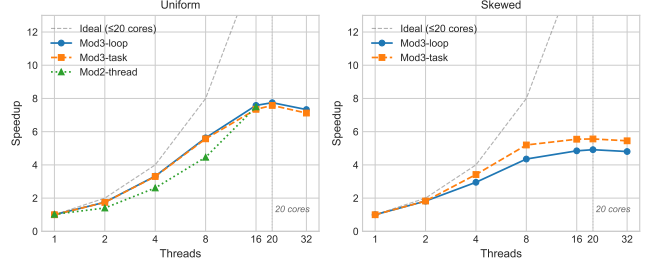


Figure 1: Strong scaling. Speedup is $T(1)/T(p)$ averaged over three repetitions. The dashed line indicates the ideal.

to the same schedule up to runtime overhead, so the LPT ordering brings no benefit and the residual gap is at the level of the OpenMP-task queue overhead. The speedup grows nearly linearly up to $T = 8$ ($\sim 5.6\times$, efficiency 70%), reaches $\sim 7.5\times$ at $T = 16$, and saturates at $\sim 7.6\times$ at $T = 20$ where all physical cores are active. The time slightly increases at $T = 32$ because the additional threads share physical cores through SMT and the kernel, dominated by streaming reads and writes through the partitioned buffers, is bandwidth-limited rather than latency-limited; SMT hides latency but does not enlarge the cache bandwidth available to each core. With $T_{\text{seq}} = 0.735$ s (Table 2), the absolute speedup at $T = 20$ is $11.93\times$.

The two strategies diverge on skewed input (Figure 2). At $T = 4$ the task variant completes in 0.130 s against 0.151 s for the loop, an advantage of $\sim 14\%$ from the LPT submission of the heavy partitions. From $T = 8$ onwards the gap widens to 13–16% as the intra-partition split engages: at $T = 20$ the task variant runs the join in 48.5 ms versus 59.6 ms for the loop, a $\sim 19\%$ reduction on the dominant phase. The hot-partition split exploits the four cooperating threads inside each hot partition and bypasses the H/T structural bound that LPT alone could not. The efficiency at $T = 20$ on skewed input is $\sim 28\%$ for the task variant against $\sim 24\%$ for the loop; the loop remains stuck at the H/T bound because no external scheduling can extract parallelism that does not exist between independent partitions.

5.3 Weak Scaling

N_R scales proportionally with T , with $2 \cdot 10^6$ tuples per thread (Figure 3).

Uniform weak scaling holds well within one socket: total time grows from 0.121 s at $T = 1$ to 0.123 s at $T = 8$ (efficiency $\sim 98\%$). The slight super-linear bump at $T = 4$ ($\text{eff} \approx 1.07$) is a known cache effect: at $T=1$

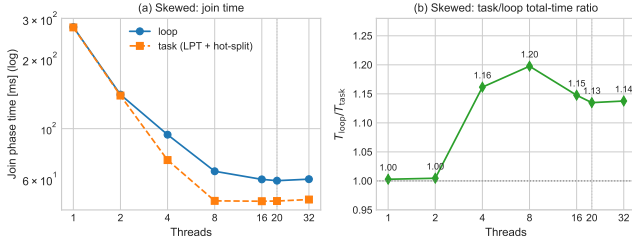


Figure 2: Skewed input. (a) Join phase time for loop and task vs T : the gap is the visible footprint of the intra-partition split. (b) Total-time ratio $T_{\text{loop}}/T_{\text{task}}$, stable around 1.13–1.17 for $T \geq 4$.

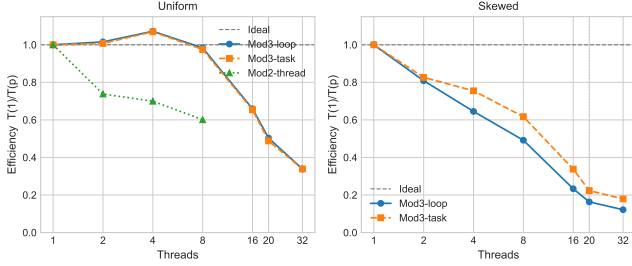


Figure 3: Weak scaling. Efficiency is $T(1)/T(p)$ with the input scaled linearly with the thread count.

a single core processes one partition group through its private caches, while at $T=4$ four cores share the 25 MB socket L3, and the per-partition **FlatCountMap** tables fit more comfortably in the aggregate cache (the same effect is documented in the Module 2 partition-count study). The fixed per-call overhead is also amortised over a $4\times$ larger run. Efficiency degrades to $\sim 66\%$ at $T = 16$ when both sockets are engaged, 50% at $T = 20$, and 34% at $T = 32$. The dominant factor is memory bandwidth: the scatter phase reads and writes a working set that grows linearly with T , and the dual-socket aggregate bandwidth saturates before the core count does. Skewed weak scaling degrades more sharply because increasing N_R also scales the load on the hot partitions in proportion; the intra-partition split mitigates this on the probe but cannot eliminate the sequential cost of the per-partition build. Skewed efficiency at $T = 16$ is approximately 23% for the loop variant and 34% for the task variant.

5.4 Phase Breakdown

The breakdown in Figure 4 reflects the memory-bound nature of the kernel. Under uniform input the join remains the dominant phase across the parallel range ($\sim 50\%$ of the total at $T = 8$, $\sim 45\text{--}48\%$ at $T=16$, around 50% at $T=20$), followed by the scatter on S ($\sim 24\%$ at $T = 8$). Under skewed input the imbalance is visible: histogram and scatter scale almost ideally because the per-tuple work does not depend on the partition distribution, while the join phase scales poorly. At $T=16$ skewed the join accounts for around 65% of the total in the loop variant and 60% in the task variant; at $T=20$ the gap is similar (66% vs 61%). The difference is attributable to the intra-partition split, which redistributes the probe load across threads inside each hot partition. We did not

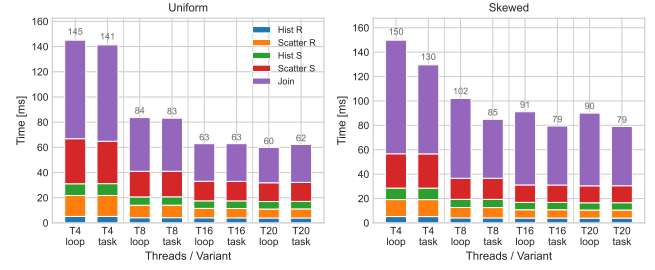


Figure 4: Phase breakdown for $T \in \{4, 8, 16, 20\}$, loop and task variants. Uniform workload on the left, skewed on the right. $T=1$ is omitted because it visually dominates and obscures the parallel regime.

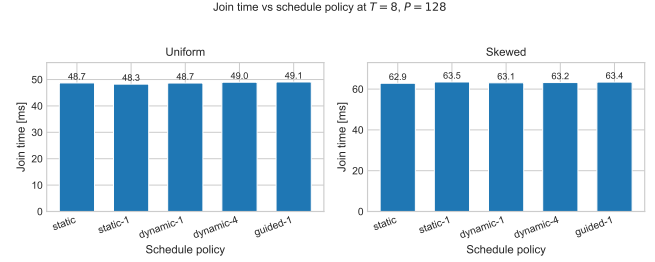


Figure 5: Join time at $T = 8$ across five schedule policies, uniform (left) and skewed (right). All policies fall within $\sim 1.5\%$ at $P/T = 16$.

extend the breakdown to $T=32$ because the SMT regime is already covered in the strong and weak scaling figures and the per-phase shares stabilise once both sockets are saturated.

5.5 Schedule Sensitivity

Figure 5 shows the join time at $T = 8$ for five schedule policies (**static**, **static,1**, **dynamic,1**, **dynamic,4**, **guided,1**) on both workloads. The policies fall within $\sim 1\%$ of each other on uniform input and within $\sim 1.5\%$ on skewed input. With $P = 128$ and $T = 8$ the ratio $P/T = 16$ is high enough that even **static** produces a balanced load: each thread receives a chunk of 16 partitions and the within-chunk variance averages out. This is the expected regime when the unit of work is small relative to the per-thread quota.

The selection of **dynamic,1** as the default is motivated by the regimes outside the measured one: at higher T and lower P/T ratios the schedule must absorb partition-size variance that **static** cannot, at the cost of a small runtime overhead which is negligible at the granularity of these experiments.

5.6 Comparison with Module 2

The Module 2 implementation uses a `std::thread` pool with `BlockingQueue` stages between phases, on the same algorithm, input, and node. Figure 6 reports time and speedup against T for the uniform workload over the full range $T \in \{1, 2, 4, 8, 16, 20, 32\}$, matching the granularity of the Module 2 evaluation. Table 2 reports the numbers at $T = 16$, $T = 20$ (physical-core saturation), and $T = 32$ (SMT range).

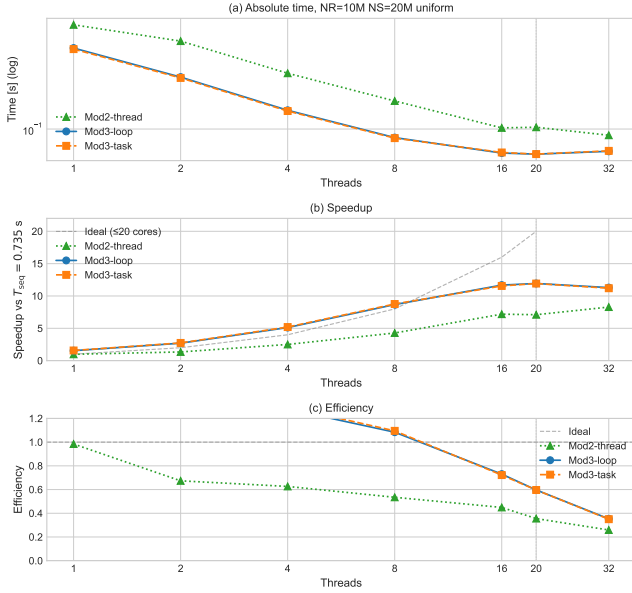


Figure 6: Module 2 `std::thread` versus Module 3 OpenMP, uniform input. (a) absolute time, (b) speedup against the shared sequential baseline, (c) efficiency = speedup/ T over the full thread range $T \in \{1, \dots, 32\}$. The vertical line marks $T=20$, physical-core saturation.

T	$T(p)$ [s]			Speedup		
	Mod2	M3 loop	M3 task	Mod2	M3 loop	M3 task
seq		0.735			1.00	
1	0.747	0.477	0.467	0.98	1.54	1.57
2	0.546	0.272	0.268	1.35	2.70	2.74
4	0.293	0.144	0.142	2.51	5.10	5.18
8	0.172	0.085	0.084	4.27	8.67	8.76
16	0.102	0.063	0.064	7.19	11.68	11.49
20	0.103	0.062	0.062	7.10	11.93	11.92
32	0.089	0.065	0.066	8.29	11.30	11.13

Table 2: Module 2 `std::thread` vs Module 3 OpenMP, uniform $N_R=10^7$, $N_S=2 \cdot 10^7$. The first row is the shared sequential baseline (0.735 s, current `hashjoin_seq` mean of 3 runs); both columns of speedups are $T_{\text{seq}}/T(p)$. The Mod2 row at $T=1$ is below unity because the `std::thread` pool already pays the cost of one worker plus a queue handshake. The optimisations of Section 2.5 apply to Module 3 only.

OpenMP outperforms the `std::thread` version at every operating point: at $T=8$ the OpenMP variants are about $2\times$ faster in absolute time, and at $T=20$ the gap is approximately $1.65\times$ (0.062 s versus 0.103 s). Two phases account for most of the absolute gap. The first is the scatter: at $T=8$ the combined scatter $R+S$ takes ~ 30 ms in OpenMP against ~ 85 ms in Module 2, a difference of 55 ms attributable to the software prefetch on the random-access destination slot. The second is the histogram: at $T=8$ it takes ~ 11 ms in OpenMP against ~ 35 ms in Module 2, where the persistent OpenMP team amortises the synchronisation cost while Module 2 pays 10–20 μ s per phase per thread for the condition-variable handshake on the blocking queue, which dominates on a phase as short as a histogram over 10^7 tuples.

In the SMT range the two implementations behave differently. Module 2 plateaus between $T=16$ and $T=$

20 (time grows from 0.102 s to 0.103 s), suggesting that the queue-based synchronisation does not scale cleanly across the second socket. The OpenMP variants continue to improve up to $T=20$ (0.062 s, $11.93\times$ against the shared baseline) and only marginally worsen at $T=32$, where the additional SMT threads share cache bandwidth that is already saturated.

The efficiency panel of Figure 6 sharpens this picture for $T > 8$, which the time and speedup curves alone tend to hide. Module 2 efficiency falls from $\sim 53\%$ at $T=8$ to $\sim 45\%$ at $T=16$ and $\sim 36\%$ at $T=20$; the OpenMP variants hold at $\sim 73\%$ at $T=16$ and $\sim 60\%$ at $T=20$. The gap widens precisely in the SMT range (the second-socket region is the relevant one for the comparison) and is the per-thread footprint of the persistent OpenMP team versus the per-phase queue handshake of the Module 2 pool.

The two OpenMP curves sit slightly above unity for $T \leq 4$ ($\text{eff} \approx 1.27$ at $T=4$). This is not super-linear scaling in the strict sense: the optimisations of Section 2.5 (software prefetch on scatter and probe, first-touch placement of the partition buffers) are present in the parallel binary but not in `hashjoin_seq`, so already at $T=1$ the OpenMP run is $\sim 1.54\times$ faster than the sequential baseline (Table 2). The numerator of the efficiency therefore carries a constant offset that decays as T grows and synchronisation cost takes over.

The two implementations also differ substantially in code volume. Module 2 includes the explicit `BlockingQueue`, the worker loop, and the manual join, which together amount to roughly twice the source of the OpenMP version. The OpenMP code expresses the same algorithm through a small number of pragmas. Explicit thread management retains its value when the structure of the computation does not match the canonical parallel-for or task-pool patterns, but the partitioned hash join falls within those patterns and benefits from the OpenMP runtime.

6 Discussion

On uniform input the two OpenMP strategies are equivalent within measurement noise; the choice between them does not affect performance. On skewed input the task variant retains a stable advantage of 12–17% across the full thread range, produced by the combination of LPT ordering at low T and intra-partition splitting of hot partitions at $T \geq 8$. The schedule-policy study shows a spread of about 1% across five options, which is the expected outcome at $P/T=16$. Across the entire thread range, the OpenMP implementation outperforms the Module 2 `std::thread` version against the shared sequential baseline: at physical-core saturation ($T=20$) the OpenMP variants reach $11.93\times$ speedup against $7.10\times$ for Module 2.

Table 3 consolidates the headline numbers at the operating points of interest.

Across the measured operating points the OpenMP implementation is faster, more compact, and more scalable than the Module 2 `std::thread` version on the same algorithm and input, with the residual margin sitting on

Metric	Uniform	Skewed (loop)	Skewed (task)
Best T	20	20	20
$T(p)$ [s]	0.062	0.091	0.080
Speedup vs T_{seq}	11.93	4.93	5.55
Efficiency at $T=20$	60%	25%	28%
$T_{\text{loop}}/T_{\text{task}}$ at $T=20$	—	1.13	

Table 3: Summary of best operating points. The shared sequential baseline is $T_{\text{seq}} = 0.735$ s.

memory bandwidth rather than on synchronisation or load balance.